La méthode « diviser pour régner »

✎ Capacités attendues

✓ Écrire un algorithme utilisant la méthode « diviser pour régner ».

Exercice 1

Le fichier `dico.txt` contient une liste de mots de la langue française, triés dans l'ordre lexicographique.

1. (a) Télécharger ce fichier et exécuter le code suivant pour créer une liste contenant les mots qu'il contient.

```
f = open("dico.txt", "r", encoding="utf-8")

dico = []
for ligne in f:
    dico.append(ligne[:-1])
```

   (b) Combien de mots ce fichier contient-il ?

2. On fournit également un fichier `mots.txt` qui contient 20000 mots. À l'exception de quelques uns, tous ces mots sont contenus dans le fichier `dico.txt`.
   Écrire un script python qui permet d'afficher ces mystérieux mots en moins de deux secondes.

Exercice 2

1. Concevoir une fonction `est_trie(t)` qui prend en argument un tableau `t` et renvoie la valeur `True` lorsque le tableau `t` est trié dans l'ordre croissant, et renvoie `False` sinon.
2. Dans cette question, aucune « pythonnade » n'est autorisé (en particulier, pas de slice).
   (a) Écrire une fonction `partie_gauche(t)` qui prend en argument un tableau `t` de taille n et renvoie le tableau constitué des `n//2` premiers éléments de `t` (on rappelle que l'opérateur `//` désigne la division entière).
   (b) Écrire de même une fonction `partie_droite(t)` qui prend en argument un tableau `t` de taille n et renvoie le tableau constitué des `n-n//2` derniers éléments de `t`.
3. On souhaite disposer d'une fonction `fusion` qui prend en argument deux tableaux `t1` et `t2` et renvoie un nouveau tableau composé de tous les éléments de `t1` et `t2` triés dans l'ordre croissant.
   On suppose que les tableaux `t1` et `t2` sont **déjà triés** dans l'ordre croissant.
   Exemple : l'appel `fusion([2, 5, 7, 9], [3, 8, 10])` renvoie le tableau `[2, 3, 5, 7, 8, 9, 10]`.
   On impose de plus les contraintes suivantes :
   - les tableaux `t1` et `t2` ne doivent pas être modifiées ;
   - le coût en nombre de comparaisons doit être de l'ordre de `max(len(t1), len(t2))` ;
   - corollaire de la contrainte précédente : l'utilisation d'un algorithme de tri est interdit.

Compléter le code de la fonctions `fusion` ci-dessous.

```
def fusion(t1, t2):
    assert est_trie(t1), "le tableau t1 n'est pas trié"
    assert est_trie(t2), "le tableau t2 n'est pas trié"
    t = []
    # TODO
    assert est_triee(t), "le tableau t n'est pas trié"
    return t
```

4. Le principe du tri fusion (*merge sort* en anglais) est le suivant :
   - découper le tableau en deux sous-tableaux `t1` et `t2` de « même » taille ;
   - trier chacun des tableaux `t1` et `t2` selon le même principe (un tableau vide ou de taille 1 est déjà trié) ;
   - fusionner les tableaux triés `t1` et `t2` grâce à la fonction `fusion` précédente.

   (a) Sur papier, appliquer le principe du tri fusion au tableau `[8, 3, 2, 1, 5, 4, 7, 6]`.

   (b) Écrire une fonction récursive `tri_fusion_rec(t)` qui utilise ce principe pour renvoyer un nouveau tableau contenant tous les éléments de `t` triés dans l'ordre croissant.
5. Écrire une version itérative `tri_fusion_it(t)` du tri fusion.

Exercice 3

On souhaite calculer x^n, où n est un entier $\geqslant 1$ et où x peut désigner un nombre réel, une fraction, une matrice et plus généralement tout élément d'un ensemble muni d'une loi de multiplication.

1. L'approche par « force brute » consiste à écrire que $x^n = \underbrace{x \cdots x}_{n \text{ facteurs}}$, et conduit à la fonction suivante.

```
def power_bf(x, n):
    p = 1
    for i in range(n):
        p = p*x
    return p
```

   Dans cet algorithme, combien de multiplications sont effectuées pour calculer x^n ?
2. Les propriétés des puissances permettent de remarquer que

$$x^n = \begin{cases} x & \text{si } n = 1 \\ x^{n/2} \cdot x^{n/2} & \text{si } n \text{ est pair} \\ x \cdot x^{n-1} & \text{si } n \text{ est impair} \end{cases}$$

   La méthode « diviser pour régner » peut donc s'appliquer dans ce contexte, et il mène à l'algorithme appelé **exponentiation rapide**, dont on donne ci-dessous une version récursive particulièrement maladroite.

```
def power_dac(x, n):
    if n==1 :
        return x
    if n%2==0 :
        return power_dac(x, n//2)*power_dac(x, n//2)
    else :
        return x*power_dac(x, n-1)
```

   (a) Expliquer pourquoi la ligne 5 est, quoique correcte, une mauvaise idée en termes de performances.

   (b) Corriger cet algorithme en conséquence.

   (c) Déterminer le coût, en nombre de multiplications, de l'algorithme corrigé pour $n = 100$.

✎ **Principe**

La méthode « diviser pour régner » (*divide and conquer*) consiste à « diviser » un problème en sous-problèmes indépendants dont les solutions sont recombinées :

- on décompose le problème en sous-problèmes ;
- on résout chaque sous-problème ;
- on fusionne les solutions des sous-problèmes en une solution globale.

La méthode « diviser pour régner » se prête particulièrement bien à une approche récursive (les sous-problèmes sont résolus grâce à des appels récursifs). Mais rappelons qu'il est toujours possible de réécrire un algorithme récursif dans une version itérative.

Lorsque la taille des entrées est grande, la méthode « diviser pour régner » permet en général d'obtenir des algorithmes plus performants (en temps d'exécution ou en espace mémoire) que ceux obtenus par « force brute ».

✎ **Définition**

Si un entier n est une puissance de 2, c'est-à-dire si $n = 2^k$ où $k \in \mathbb{N}$, alors l'exposant k est appelé logarithme (binaire) de n, noté $\log(n)$ (aussi noté $\log_2(x)$).

C'est aussi le nombre de chiffres, moins 1, de l'écriture binaire de n.
Si n n'est pas une puissance de 2, on peut ainsi définir son logarithme comme étant le nombre chiffres dans son écriture binaire.

Remarque : en toute rigueur, le logarithme binaire d'un réel $x > 0$ est défini par la formule $\log\left(x\right) = \frac{\ln(x)}{\ln(2)}$.

Exercice 4

Utiliser la définition pour compléter le tableau suivant avec des valeurs approchées entières.

n	15	33	128	1000	60 000	
$\log(n)$						24

Exercice 5

Compléter le tableau suivant et comparer les ordres de grandeur des différents modes de croissance.

n	10	100	1000	10^4	10^6	10^9
$\log(n)$						
n						
$n\log(n)$						
n^2						

✎ **Proposition**

Le coût moyen, en nombre de comparaisons, de la recherche dichotomique dans un tableau de taille n est _____

Élément de preuve

On suppose que $n = 2^k$: chaque comparaison avec l'élément central permet de diviser l'espace de recherche par 2.

Au terme de k comparaisons, l'espace de recherche se réduit donc à un seul élément : on voit ainsi que le coût en comparaisons est de l'ordre de $k = \log(n)$ comparaisons.

✎ **Proposition**

Le coût moyen, en nombre de comparaisons, du tri fusion d'un tableau de taille n est _____

Élément de preuve

On suppose que $n = 2^k$ et on note c_k le coût du tri d'un tableau de taille 2^k. En analysant l'algorithme du tri fusion, on peut exprimer c_k en fonction de c_{k-1}

$$c_k = \underbrace{2c_{k-1}}_{\text{coût des appels récursifs}} + \underbrace{2^k}_{\text{coût de la fusion}}$$

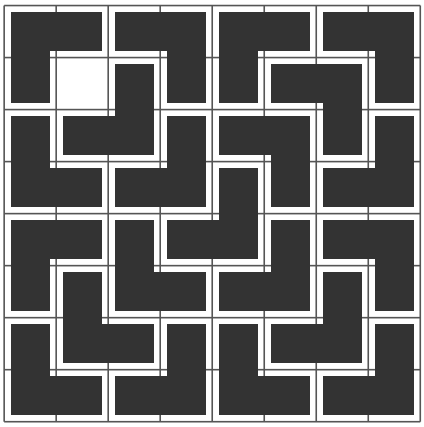
et on peut vérifier que l'on a bien $c_k = 2^k \times k = n\log\left(n\right)$.

Exercice 6

Sur une grille carrée comportant $2^n \times 2^n$ cases, on souhaite disposer les motifs ci-dessous pour qu'ils recouvrent la grille, à l'exception d'une seule case (il n'est pas possible de recouvrir parfaitement la grille avec ces motifs).



Exemple de recouvrement avec $n = 3$.



Écrire un algorithme récursif s'appuyant sur une stratégie « diviser pour régner » qui permette, étant donnée une case repérée par ses coordonnées `(i, j)`, de recouvrir un échiquier carré de dimensions $2^k \times 2^k$ par les motifs ci-dessus.

Exercice 7

L'image `turnbuffon.png` de dimensions 512×512 pixels a été tournée d'un quart de tour.
On s'intéresse dans cet exercice à deux manières différentes d'afficher l'image dans son orientation originale.
On fournit pour cela, un fichier `turnbuffon.py`, qui affiche l'image dans une fenêtre *pygame*, et permet de la modifier à l'aide des fonctions suivantes.

- `get_pixel(i, j)` : renvoie la valeur du pixel de coordonnées (i, j)
- `set_pixel(i, j, val)` : définit la valeur du pixel de coordonnées (i, j)
- `update()` : rafraîchit l'affichage
- `execute(f)` : exécute la fonction `f` durant l'affichage.

Exemple d'utilisation

```
from turnbuffon import *

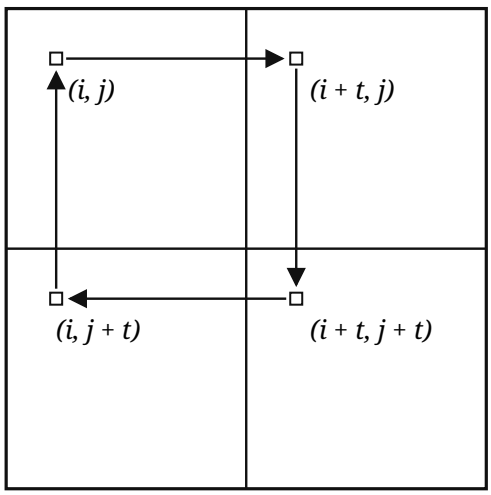
def rotation():
    for i in range(512):
        p = get_pixel(i, 0)
        for j in range(200, 300):
            set_pixel(i, j, p)
        update()

execute(rotation)
```

1. (a) Écrire une fonction `rotation_tab` pour que l'image soit affichée dans son orientation originale. Pour cela, on stockera dans un tableau l'ensemble des valeurs des pixels, qui seront ensuite affichés dans le bon ordre.

   (b) Sachant qu'un pixel est représenté par quatre nombres entiers compris entre 0 et 255 (rouge, vert, bleu et transparence), estimer le coût en mémoire (en Mo) de cette méthode `rotation`.

2. On souhaite mettre en œuvre une solution s'appuyant sur le paradigme « diviser pour régner » dont le coût en mémoire est négligeable. Pour une image carrée dont le côté est une puissance de 2, le principe est le suivant :
   - on découpe l'image en quatre carrés de côté t ;
   - si $t > 1$, on applique récursivement la méthode `rotation` à chacun de ces carrés ;
   - on permute ensuite les valeurs des pixels de ces quatre carrés selon le schéma ci-dessous.



   En utilisant ce principe, écrire le code d'une fonction `rotation(x=0, y=0, t=512)` pour qu'elle effectue la rotation d'un quart de tour du carré de côté `t` dont le coin supérieur droit a pour coordonnées `x` et `y`.

Exercice 8

Étant donnés n points du plan, on cherche à déterminer quelle est la plus petite distance entre deux de ces points.
On supposera dans la suite que les points sont donnés sous la forme d'un tableau de tuples.
En particulier, le code ci-dessous permet de générer une liste de 1000 points dont les coordonnées sont tirées au hasard dans l'intervalle $\left[0; 10^9\right]$.

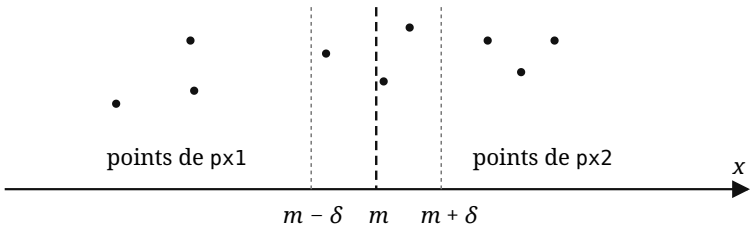
```
from random import random
points = [ (random()*10**9, random()*10**9) for _ in range(1000)]
```

1. On aura besoin d'une fonction `distance(p, q)` qui renvoie la distance entre deux points, les tuples `p` et `q` représentant leurs coordonnées : écrire cette fonction.
2. (a) Écrire une fonction `shortest_distance_bf(pts)` qui renvoie la plus courte distance entre deux points du tableau `pts` en utilisant une approche par « force brute ».
   (b) Estimer le nombre d'appels à la fonction `distance` en fonction de la taille n du tableau `pts`.
3. Ce problème se prête à une approche « diviser pour régner », que l'on présente dans cette partie.
   Tout d'abord, la stratégie suppose que l'on dispose de deux tableaux `px` et `py` qui contiennent les coordonnées des points triés respectivement selon leur abscisse croissante et leur ordonnée croissante.

   Le principe de l'algorithme est alors le suivant.

   - On coupe le tableau `px` en deux sous-tableaux `px1` et `px2` de « même taille ».
   - On applique récursivement l'algorithme pour déterminer la plus petite distance d_1 entre deux des points de `px1`, et la plus petite distance d_2 entre deux des points de `px2`.
   - La plus petite distance $d_{\min}$ entre deux des points de `px` est alors atteinte dans l'un des cas suivants :
     – entre deux points de `px1`, auquel cas $d_{\min} = d_1$;
     – entre deux points de `px2`, auquel cas $d_{\min} = d_2$;
     – entre un point de `px1` et un point de `px2`.

   Il faut donc déterminer la plus petite distance entre les points de `px1` et les points de `px2`.
   Pour cela, on note m la moyenne des abscisses du dernier élément de `px1` et du premier élément de `px2` : le tableau `px` étant trié, tous les points de `px1` ont une abscisse inférieure à m, tandis que tous les points de `px2` ont une abscisse supérieure à m. En notant $\delta = \min(d_1, d_2)$, la recherche de la plus courte distance peut alors être limitée aux points dont les abscisses sont comprises entre $m - \delta$ et $m + \delta$.



   En sélectionnant les points du tableau `py` dont les abscisses sont comprises entre $m - \delta$ et $m + \delta$, il ne reste plus qu'à faire la remarque cruciale suivante : la différence entre les ordonnées des éléments d'indice $i + 7$ et d'indice i est supérieure à δ.

   (a) Écrire une fonction `get_sorted(pts)` qui renvoie les tableaux triés `px` et `py`.
   (b) Écrire une fonction récursive `shortest_distance_dac` qui prend en argument `px` et `py`, et met en œuvre la méthode proposée ci-dessus (`dac` : *divide and conquer*).
   (c) La fonction obtenue est-elle plus performante que la fonction `shortest_distance_bf` ?
   Optimiser l'implémentation pour que ce soit bien le cas.